Компьютерная графика

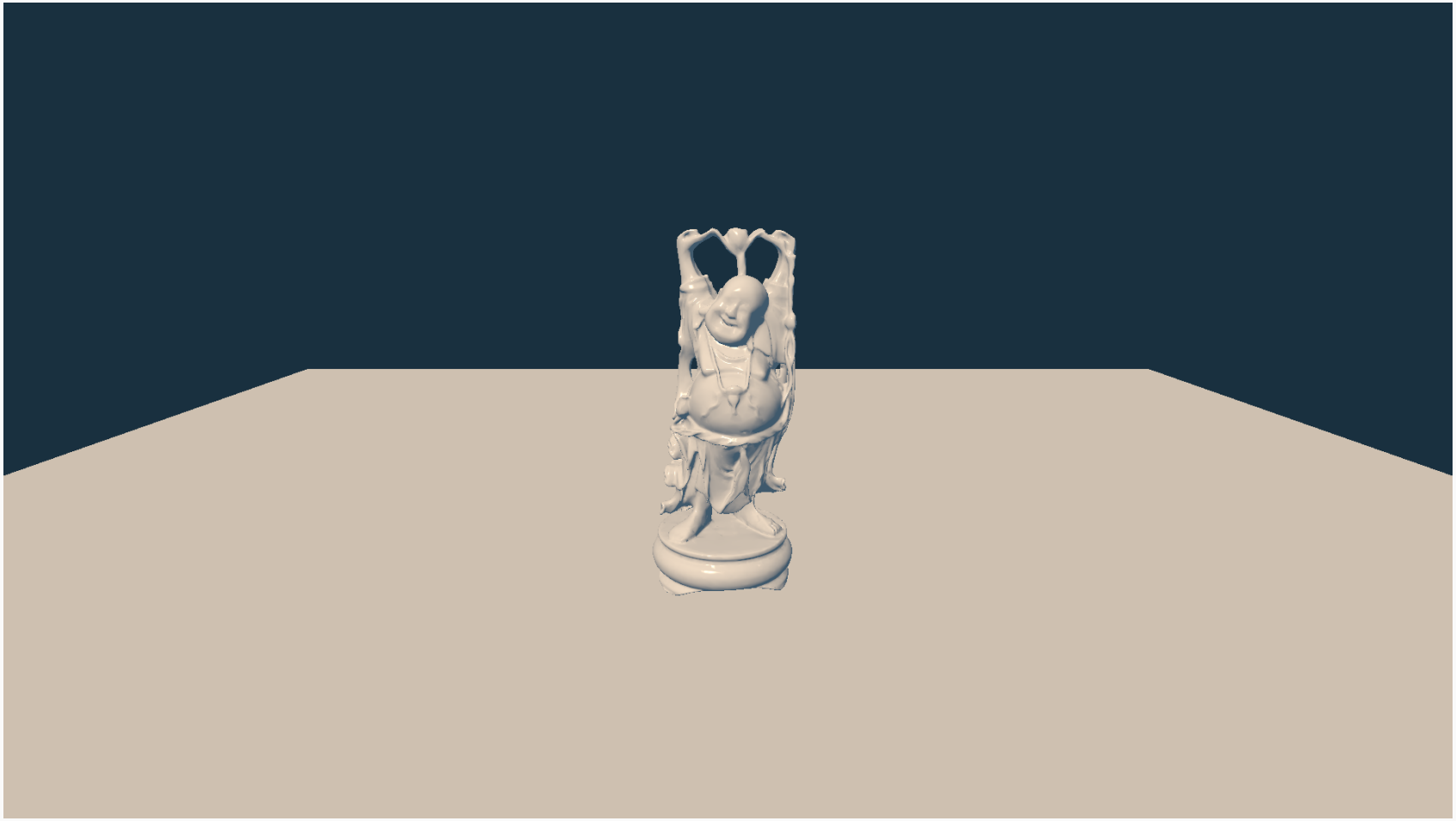
Практика 8: Shadow mapping

2026

Практика 8

- Камеру можно двигать и вращать стрелками

Практика 8

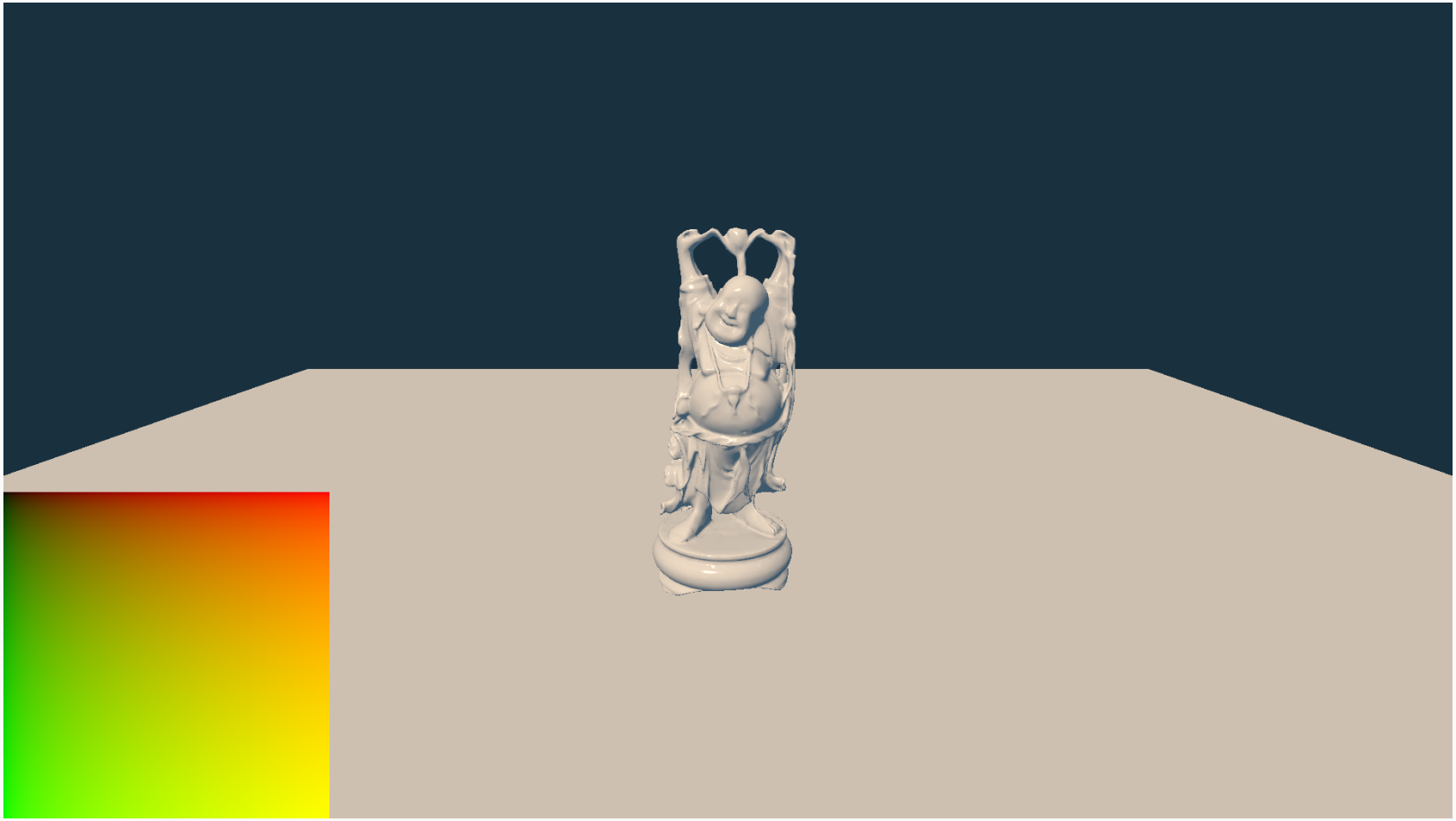


Задание 1

Выводим дебажный прямоугольник для shadow map

- В практике уже есть `debug_depth_texture_pipeline`
- Используем её, чтобы нарисовать дебажный **квадрат** в левом нижнем углу
- Для этого нужно правильно выставить `DebugDepthTextureImmediates`
- `draw(vertex_count = 6)`
- **NB:** квадрат должен оставаться квадратом при изменении размеров окна!

Задание 1



Задание 2

Создаём shadow map и sampler

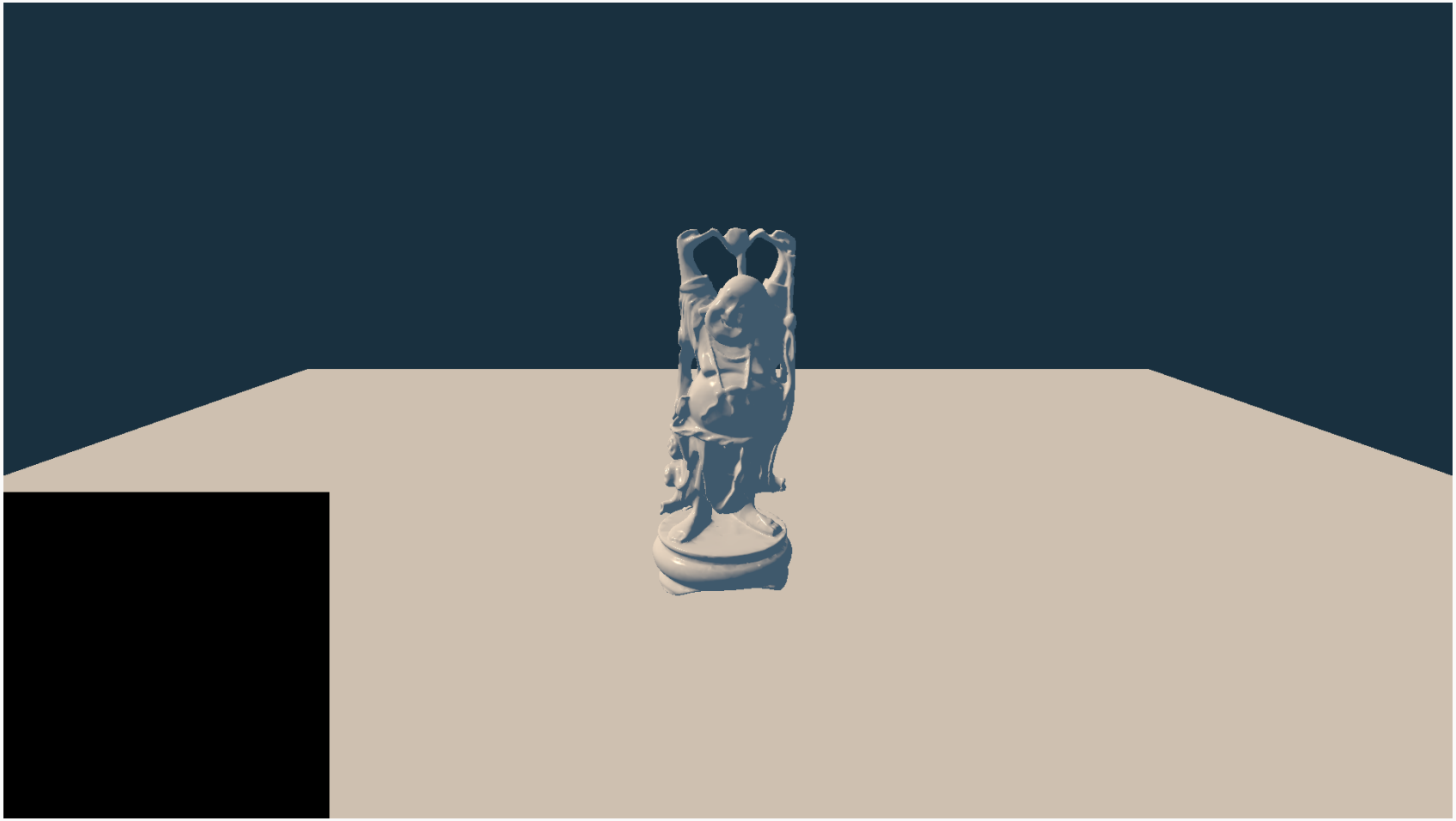
- Создаём текстуру shadow map: 1024×1024, `usage = RENDER_ATTACHMENT | TEXTURE_BINDING`, `format = Depth24Plus`
- Создаём sampler: `address_mode = CLAMP_TO_EDGE`, `filter = NEAREST`
- Добавляем их в lights bind group как binding 1 и 2, соответственно
- Для bind group layout: `texture.sample_type = Depth`, `sampler = Filtering`,
- **NB:** Пока ничего не изменится, так как мы не используем shadow map

Задание 3

Читаем shadow map в дебажном квадрате

- Добавляем shadow map и sampler как bindings в `debug_depth_texture_shader.wgsl`
- Для shadow map тип – `texture_depth_2d`
- Во фрагментном шейдере читаем shadow map `textureSample(shadowMap, shadowMapSampler, in.texcoord)` и выводим его как цвет
- **NB:** Пока там записаны нулевые/мусорные значения – мы ещё ничего не записали в shadow map

Задание 3

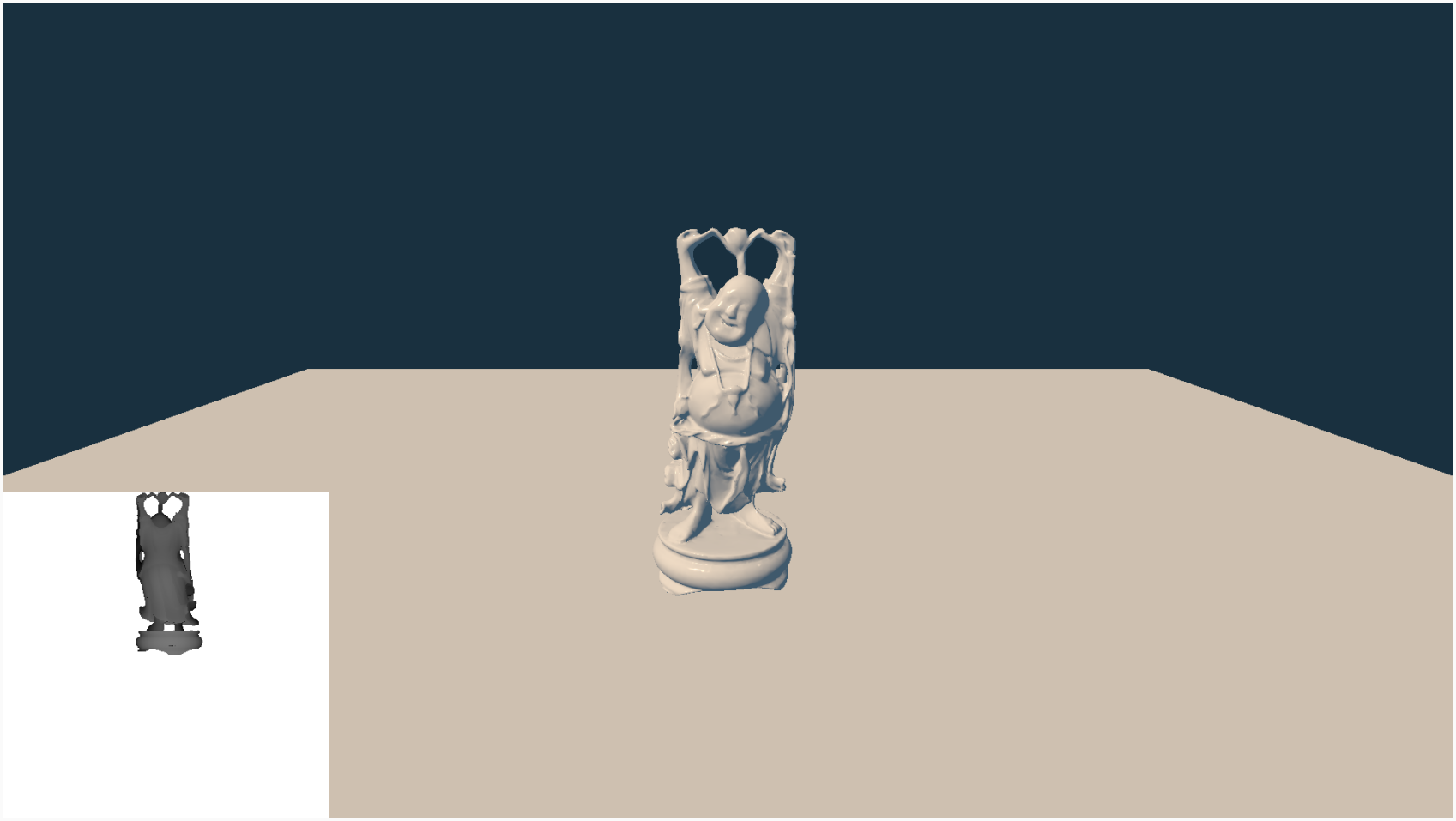


Задание 4

Генерируем shadow map

- Создаём новый шейдер для генерации теней: он просто записывает `position` входной вершины как `@builtin(position)` выходного значения шейдера (другие атрибуты ему не нужны)
- Фрагментный шейдер тут тоже не нужен :)
- Создаём новый render pipeline для генерации теней: он использует новый шейдер, те же настройки depth-stencil, но не выводит никакой цвет (fragment state = null)
- Bind группы ему тоже не нужны
- Заводим новый render pass **до** основного: у него есть текстура shadow map в качестве depth attachment (`store_op = Store`), но нет ни одного color attachment
- В этом render pass рисуем нашу 3D модель используя новый render pipeline (`set_vertex_buffer()`, `set_index_buffer()`, `draw_indexed()`)
- Модель должна появиться в дебажном квадрате

Задание 4

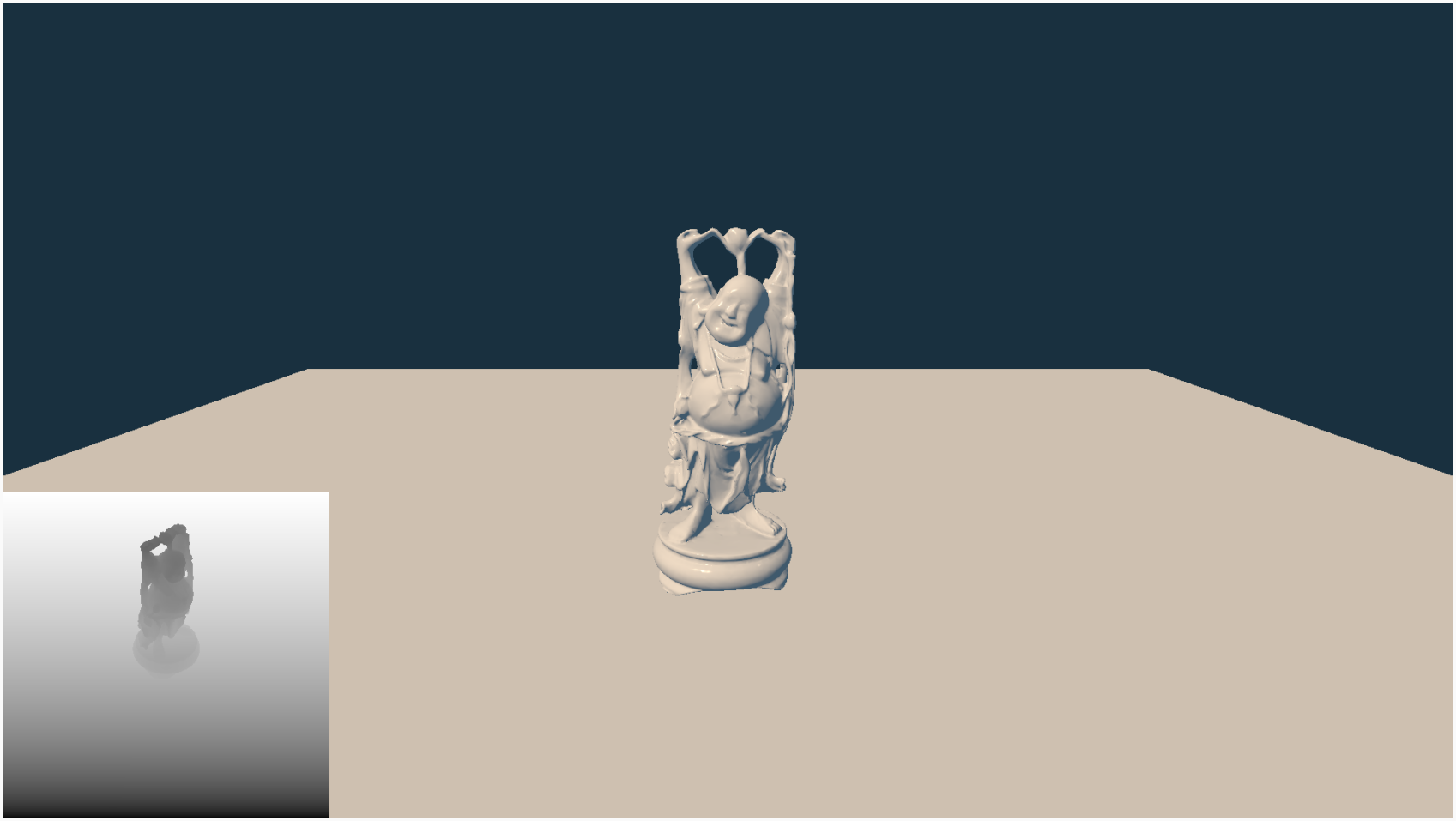


Задание 5

Вычисляем проекцию для shadow map

- Заводим оси проекции:
  - `Z = -light_direction`
  - `X` и `Y` – любые, образующие с `Z` ортонормированный базис
- Эти векторы дадут **строки** матрицы перевода в систему координат теней
- Диапазон значений Z – $[0, 1]$, поэтому тени обрежутся плоскостью, проходящей через начало координат
- Это проще всего исправить, домножив проекцию теней на матрицу преобразования $z \mapsto \frac{z+1}{2}$
- Получившуюся матрицу нужно передать в шейдер генерации теней (проще всего как immediate) и применить к входным вершинам
- Shadow map в дебажном прямоугольнике должна начать двигаться со временем

Задание 5

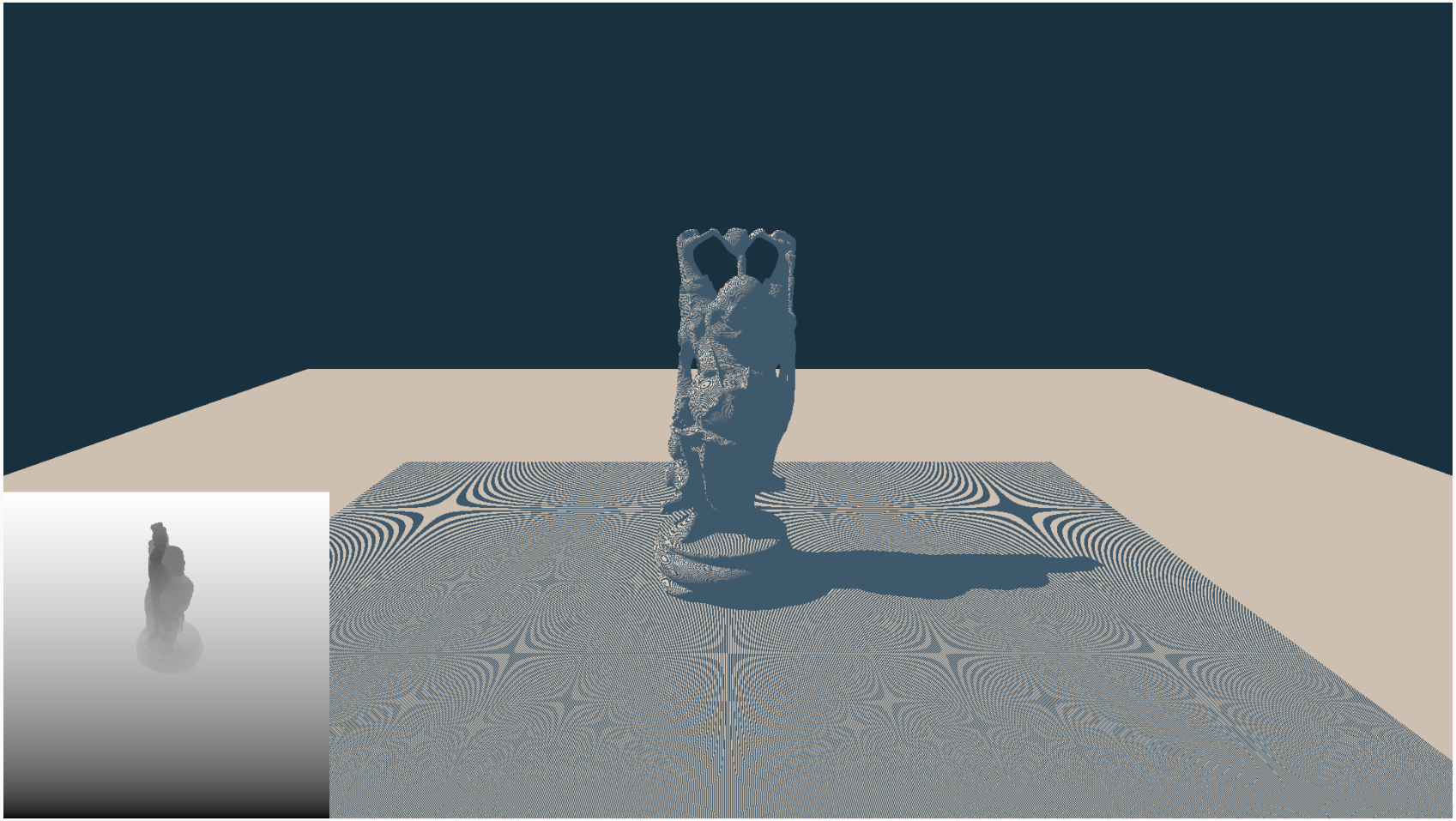


Задание 6

Накладываем тени

- В lights uniforms добавляем матрицу shadow projection, вычисленную на предыдущем шаге
- В основной шейдер добавляем bindings для shadow map и sampler
- Во фрагментном шейдере вычисляем, находится ли в тени данный пиксель (см. слайды лекции)
- Применяем освещение от источника света только если мы **не находимся** в тени (ambient применяем всегда!)
- **NB:** Будет очень сильный shadow acne

Задание 6

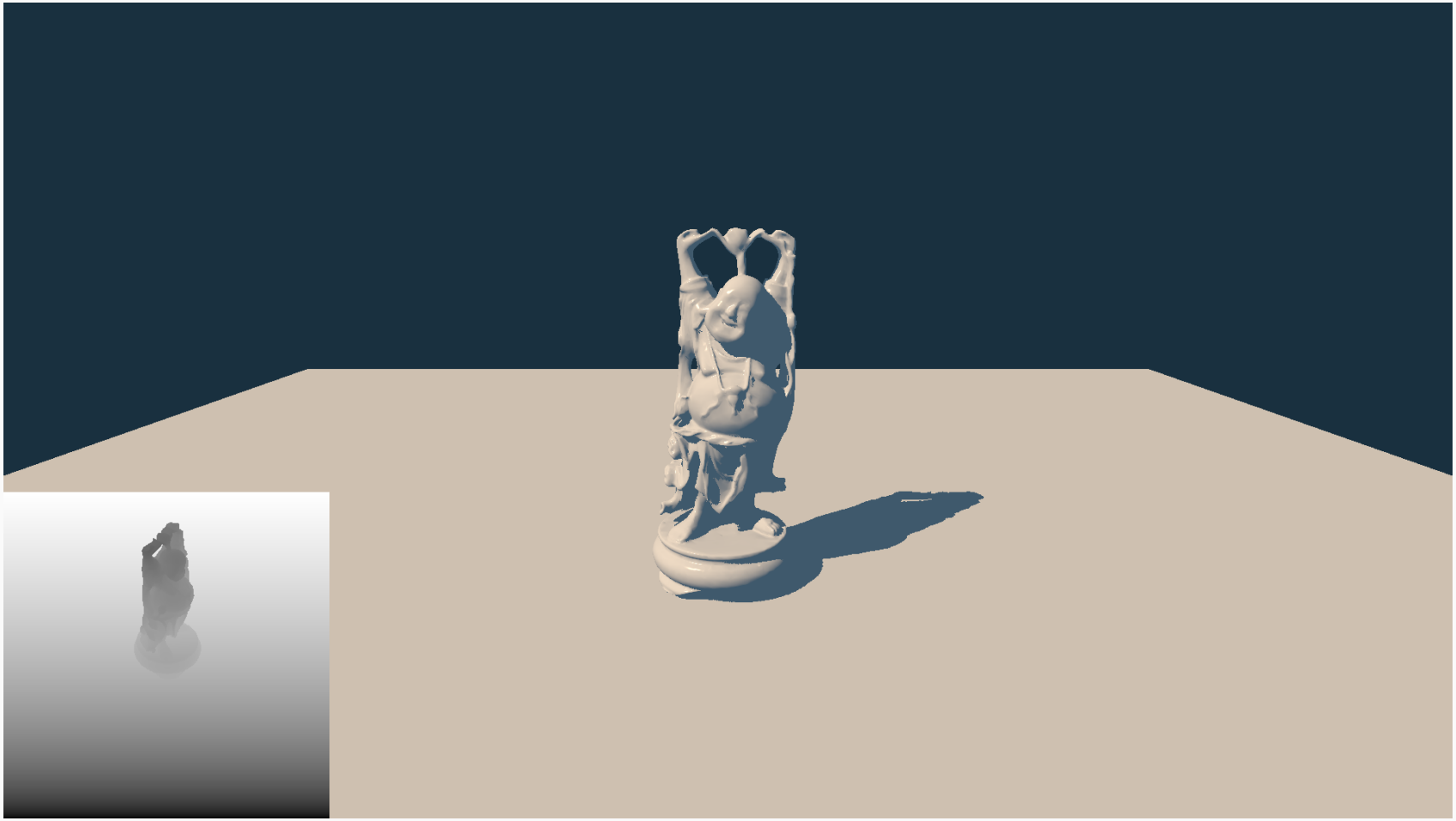


Задание 7

Добавляем shadow bias

- Вычитаем маленькое значение из `shadow_coord.z`
- Значение нужно подобрать, чтобы исчез shadow acne, но чтобы тени не съехали слишком сильно

Задание 7



Задание 8*

Добавляем PCF

- Создаём ещё один sampler, `filter = Linear`, `compare = Less` (старый sampler нельзя переиспользовать – сломается дебажная визуализация shadow map)
- Добавляем его в lights bind group, `sampler = Comparison`
- Добавляем соответствующий binding в основной шейдере, тип sampler'а – `sampler_comparison`
- При наложении теней в основном шейдере используем не `textureSample`, а `textureSampleCompare` – она вернёт **множитель**, на который нужно домножить освещение от источника света
- Вблизи тени должны стать гладкими

Задание 8*

